Interview Task: Job Queue Service (Mid/Senior Backend Engineer)

Space: Engineering Hiring **Version:** 1.0 **Last Updated:** May 2026

interview-task mid-senior backend Python

> This assignment evaluates your architectural and engineering skills. Use AI tools, but understand every line of your code.

Welcome

Thank you for participating in our technical assessment process. We're excited to see what you build!

This project is designed to evaluate your ability to design and implement distributed systems - the kind of challenges you'd face working on our backend infrastructure. Take your time, think through the edge cases, and show us how you approach complex problems.

About This Assessment

You'll be building a distributed background job processing system. This is a core infrastructure component used in many production systems for handling async tasks like sending emails, processing uploads, generating reports, and more.

We're interested in seeing how you handle concurrency, failure scenarios, and state management. There's no single "right" answer - we want to see your thought process and trade-offs.

Using AI Tools

You are **explicitly permitted and encouraged** to use AI coding assistants during this assessment. This includes ChatGPT, GitHub Copilot, Claude, Cursor, or any other AI tools you prefer.

Why we allow this:

- AI tools are part of modern development - we use them too

- Distributed systems have subtle edge cases that AI often gets wrong
- We're evaluating your ability to critically assess AI suggestions

What we're actually evaluating:

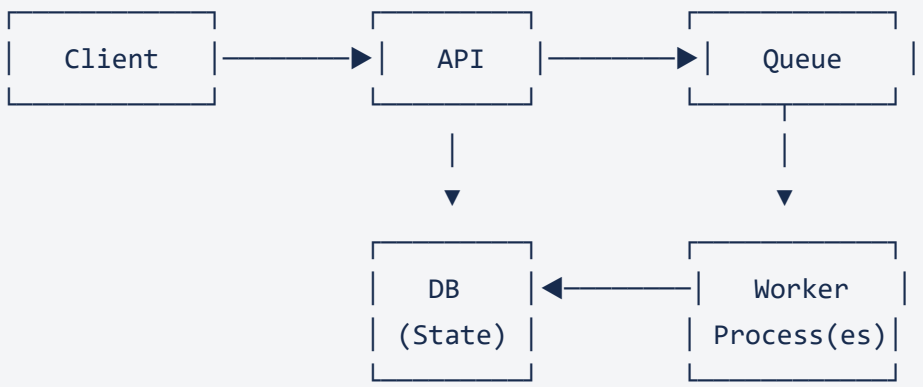
- Do you understand the concurrency challenges in this problem?
- Can you identify and handle failure scenarios?
- Do you recognize when AI gives you subtly incorrect distributed systems advice?
- Can you explain the trade-offs in your design?

Important: You will be asked to explain your architecture and handle "what if" scenarios in a follow-up interview. Make sure you deeply understand your solution.

System Overview

Build a job queue system with these components:

1. **API Service** - Accepts job submissions and status queries
2. **Queue** - Holds pending jobs (use any queue/cache technology)
3. **Worker Process(es)** - Pull jobs and execute them
4. **Database** - Stores job state and results



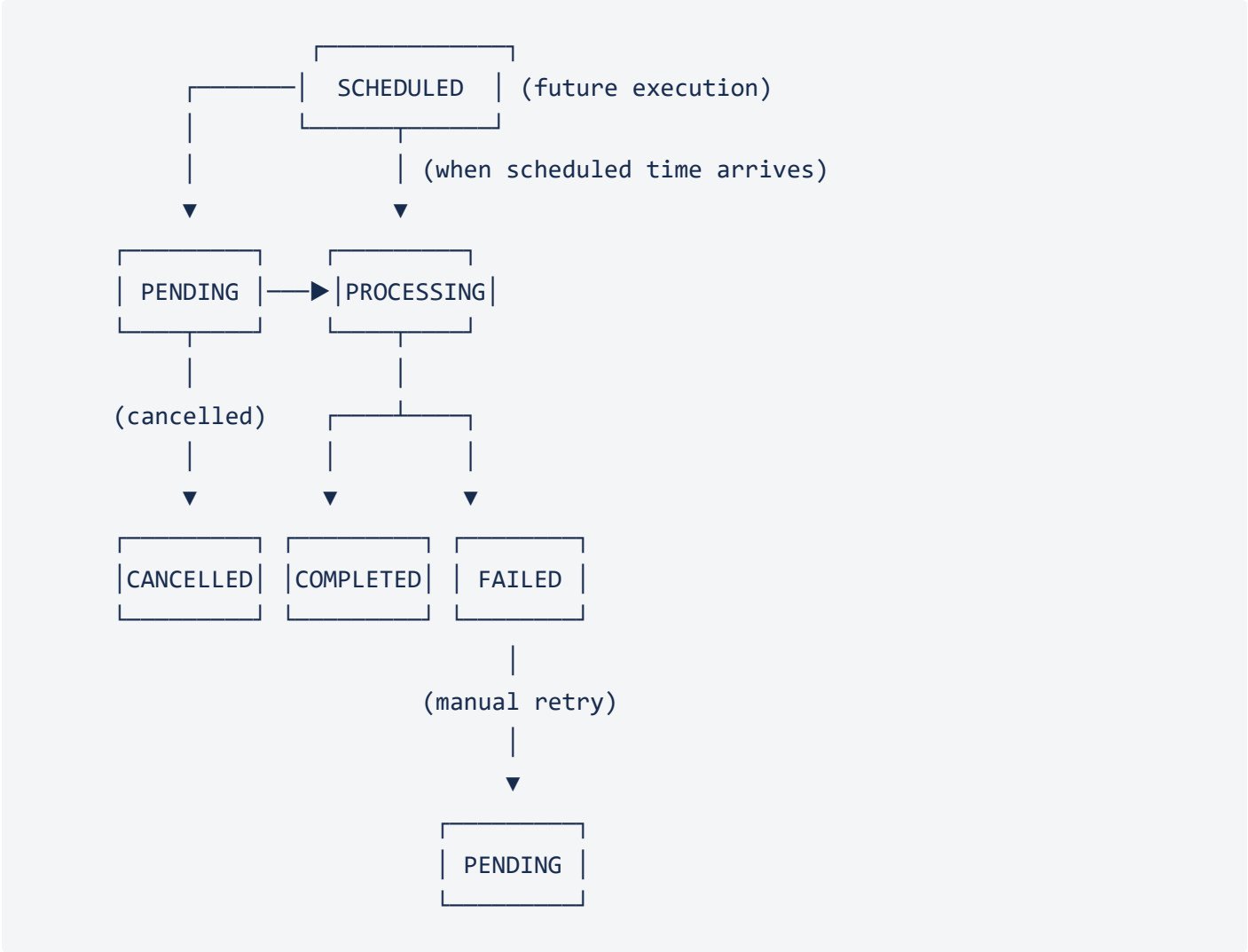
Functional Requirements

API Endpoints

- Submit a new job

- Get job status, result, or error
- List jobs with filters (status, type)
- Cancel a pending/scheduled job
- Retry a failed job
- Health check with queue statistics

Job Lifecycle



Job Types to Implement

Implement these mock job types to demonstrate your system:

1. Email Job

Simulates sending an email. Sleep 1-3 seconds, return success with mock message ID.

2. Webhook Job

Simulates calling an external webhook. Sleep 1-2 seconds. 80% success rate, 20% simulated failure (for testing retry logic).

3. Report Job

Simulates generating a report. Sleep 3-5 seconds, return mock file URL.

4. Batch Job

Processes multiple items. Process each item with small delay, track progress percentage, return summary.

Data Model

Design your data model to support:

Job

- Unique identifier
- Job type and payload (JSON)
- Status (scheduled, pending, processing, completed, failed, cancelled)
- Priority level (higher = more urgent)
- Attempt tracking (current attempts, max attempts)
- Error information for failed jobs
- Progress percentage (for batch jobs)
- Scheduling (optional future execution time)
- Timestamps (created, started, completed)
- Result storage (JSON)
- Idempotency key (for duplicate prevention)

Job Log (optional but recommended)

- Associated job
- Log level (info, warning, error)

- Message and metadata
- Timestamp

Critical Implementation Details

1. Job Pickup - Preventing Duplicates

When multiple workers are running, you must ensure a job is only picked up by one worker.

Consider: Atomic queue operations, database locking, lease-based claiming

Document your approach in DECISIONS.md

2. Worker Crash Recovery

If a worker crashes while processing a job, the job shouldn't be stuck in PROCESSING forever.

Consider: Heartbeats, job timeouts, visibility timeouts, background monitors

Document your approach in DECISIONS.md

3. Retry with Backoff

Failed jobs should retry with exponential backoff:

- Attempt 1: immediate (or nearly immediate)
- Attempt 2: after delay (e.g., 30 seconds)
- Attempt 3: after longer delay (e.g., 2 minutes)
- After max attempts: mark as FAILED permanently

4. Priority Queue

Higher priority jobs should be processed before lower priority jobs.

5. Scheduled Jobs

Jobs with a future execution time should not be processed until that time arrives.

6. Idempotency

If a job is submitted with the same idempotency key:

- Return the existing job (don't create duplicate)
- Store idempotency keys for at least 24 hours

Technical Requirements

Must Have

- Python 3.11+
- Web framework of your choice
- Relational database for job persistence
- Queue/cache technology for job queue
- Separate worker process (not in-request processing)
- Retry with exponential backoff (3 attempts)
- Priority-based processing
- Job cancellation
- Containerized setup (docker-compose runs both API and worker)
- At least 6 meaningful tests:
- Job submission and retrieval
- Job completion flow
- Job failure and retry
- Cancellation
- Idempotency
- Priority ordering

Should Have

- Scheduled job support
- Worker crash recovery mechanism
- Structured JSON logging with job context
- Graceful shutdown (finish current job before exit)

- Health endpoint with queue stats

Nice to Have

- Multiple concurrent workers
- Progress tracking for batch jobs
- Job timeout enforcement
- Dead letter queue for poison messages

Submission Requirements

Submit a Git repository containing:

```
your-project/
├── README.md
├── AI_USAGE.md
├── DECISIONS.md
├── docker-compose.yml
├── Dockerfile
├── (your application code)
└── (your tests)
```

README.md

Include:

1. How to run the project
2. How to run tests
3. How to submit a test job (example request)
4. Brief architecture overview

DECISIONS.md (Required)

```
# Design Decisions

## 1. Job Pickup Strategy
```

```
**Approach chosen:** [Your approach]

**Why:** [Your reasoning]

**Trade-offs:** [What you gave up, what you gained]

---

## 2. Worker Crash Recovery

**Approach chosen:** [Your approach]

**Why:** [Your reasoning]

**What happens if worker crashes mid-job:** [Explain the recovery flow]

---

## 3. Priority Queue Implementation

**Approach chosen:** [Your approach]

**Why:** [Your reasoning]

---

## 4. Retry Backoff Strategy

**Approach chosen:** [Your approach]

**Timing:** [Specific delays you chose]

---

## 5. One Thing I Would Do Differently With More Time

[Be honest - what did you skip or simplify?]
```

AI_USAGE.md

```
# AI Tool Usage

## Tools I Used
[List the AI tools you used]

## What Helped Most
[Describe 1-2 specific cases where AI helped significantly]

## What I Had to Fix
[Describe 1-2 cases where AI gave incorrect advice - especially around concurrency]

## What AI Struggled With
[Any parts where AI wasn't helpful]
```

Tips for Success

1. **Get the basic flow working first**: submit → queue → process → complete
2. **Add failure handling second**: retries, error storage
3. **Tackle concurrency last**: it's the hardest part
4. **Be honest in DECISIONS.md**: we value self-awareness over perfection
5. **Test the worker independently**: it's easy to miss bugs there

---

We look forward to seeing your solution and discussing it with you!